

HelixCluster

HelixCluster

データセンターGPUからエッジ携帯端末まで、単一のコントロールプレーンで統合するAIコンピュータ向け分散オペレーティングシステム

概要

Helix Cluster OSは、次世代の分散オペレーティングシステムであり、データセンターGPUからエッジSBCや携帯端末に至るまで、異種ノード間のコンピュータを統合的にオーケストレーションします。HPCスケジューリング、コンテナオーケストレーション、AI/ML推論、フェデレーテッドマルチクラスタ運用、そしてセキュアなマルチテナントセッションを単一のコントロールプレーンの下に統合します。

短い説明

Goベースの分散OS/GPU共有コンピュータクラスタ。HPCスケジューリング（Omegaモデルの二段階スケジューラ）、コンテナオーケストレーション、AI推論ルーティング、フェデレーション、そして異種ノード間でのセキュアなマルチテナントセッションを統合し、SWIMゴシップとRaftコンセンサスによって調整されます。また、耐量子暗号によるエンドツーエンド暗号化を実装しています。

長い説明

Helix Cluster OSは、データセンターGPU、エッジシングルボードコンピュータ、さらには携帯端末といった極めて多様なハードウェア上でコンピュータワークロードをオーケストレーションし、単一のコントロールプレーンの下に統合します。A100のラックも、一握りのSBCも、互換性のない孤立した島々ではなく、一つのアドレス可能なファブリックとして扱います。これは、L0のハードウェア基盤からL7のフェデレーションとオプザバビリティまでの7層スタックを実装したGoワークスペース（モノレポ+gitサブモジュール）であり、14のコントロールプレーンマイクロサービスによって調整されます。ノードのメンバーシップはSWIMゴシップとディスカバリによって追跡され、ノードの参加や離脱に応じてファブリッ

クが自己修復します。強固な整合性を持つ状態はRaftコンセンサス上に構築され、シャードごとのRaftグループとして編成され、リースホルダーによるローカル読み取りで高速化し、STONITHフェンシングにより、分断されたノードが共有状態を破壊しないことを保証します。

ワークロード配置はOmegaモデルの二段階スケジューラを通じて行われます。楽観的同時実行制御、ClassAdマッチング、ギャングスケジューリング、バリューマルチプライヤーによるプリエンブション、制約ベースの配置を採用し、従来のHPCスケジューラを超える機能を提供します。具体的には、カーボンアウェアかつコスト／TCOアウェアなルーティング、クラウドへのバーストオートスケールリング、そしてマーケットプレイスアダプター（Akash、io.net、RunPod、AWS Spot、Chutes）を介して、ローカルリソースが不足した際にはレンタルキャパシティにジョブをスピルオーバーさせることが可能です。

エンドユーザーが直接目にするのは、このような内部機構ではありません。ユーザーは、クリーンなセッションモデル（コンピュート割り当て）、インタラクティブなWebSocket/PTYターミナル、内部AI推論ルート、そしてプール利用状況の読み取りを通じて操作します。セキュリティは後付けではなく、第一のレイヤーとして設計されています。SPIFFEアイデンティティ、デバイスアテストーション（チャレンジ／レスポンス、GPUワークの証明、シーリング）、輸出管理KYCゲート、そして耐量子エンドツーエンド暗号化トランスポートを採用。X25519とML-KEM-768のハイブリッド鍵交換にAEADレコード保護とリプレイ拒否を組み合わせ、将来の量子コンピュータによる攻撃にも耐えうる設計となっています。正確性は主張されるものではなく、実証されます。FoundationDBスタイルのシードラン、フォールトインジェクション、ネットワークシミュレーション、バイト単位のリプレイ、Porcupineリニアラビリティチェッカーを用いた決定論的シミュレーションテストにより、分散システムの障害を必要に応じて再現可能です。また、ペアミュレーションテストを義務付け、ガードテストが実際に機能することを証明します。アーキテクチャとドキュメントの整合性は、現実とドキュメントが乖離した瞬間にビルドを失敗させるメカニカルリントによって保たれます。

コンテンツ

なぜこれを作ったのか

AIやHPCワークロードを、まったく異なるハードウェア階層にまたがって実行するために——個別のスケジューラ、オーケストレータ、推論スタックを寄せ集めることなく、そして何より、出荷される機能はすべて実際のエンドユーザーの挙動を証明する（スタブ上のグリーンテストでは決してない）というエンジニアリング上の保証を持ち、またプラットフォーム固有の機能はすべてそのプラットフォームのネイティブな仕組みを用いる（Linux専用のモックなどは使わない）という設計で実現するためだ。このプロジェクトが明確に排除することを目指している問題は、リポジトリのガバナンスにも引用されている「テストは通るが機能は実際には動かない」という失敗モードである。

なぜゲームチェンジャーなのか

通常なら五つの独立したスタック——HPCスケジューリング、コンテナオーケストレーション、AI推論、マルチクラスタフェデレーション、セキュアなマルチテナントセッション——を必要とするものを、データセンターのGPUからエッジのハンドヘルドデバイスに至るまで、単一のコントロールプレーンに統合する。しかも、その実現には通常は特殊なインフラにしか割かれない厳格な予算が投じられている。形式手法レベルの正確性（TLA+仕様、決定論的シミュレーション、リニアラビリティチェック）や、量子コンピュータ後も安全な機密通信といった、ほとんどのオーケストレータが試みようとしめない保証が含まれる。技術的な差別化に加え、コストと炭素排出量を考慮した配置やクラウドマーケットプレイスを活用したバースト機能により、経済的なレバレッジとしても機能する。スケジューラは自動的に安価で環境負荷の少ない、あるいは余剰のリソースを追求するため、同じワークロードでもコストと排出量を抑えられる——誰もジョブを書き直す必要はない。

革新的な点

- 決定論的シミュレーションテスト（**DST**）——シード値を用いた完全再現可能なシミュレータで、障害、クロックスキュー、ネットワークパーティションを注入し、その結果をバイト単位で再生。さらにPorcupineリニアラビリティチェッカーにかけることで、一度発見されたハイゼンバグをいつでも再現可能にする。
- オメガモデルの二層スケジューラ——楽観的並行配置、ClassAdマッチング、ギャングスケジューリング、バリューマルチプライヤによるプリエンプレションを備えた共有状態設計。中央のボトルネックなしに、多数のスケジューラが単一クラスタに対してコミットできる。
- 耐量子**E2EE**／機密推論ワイヤリング——X25519 + ML-KEM-768のハイブリッド鍵交換で、リクエストごとにレスポンスキーペアをバインドし、AEADによるリプレイ拒否を実装（暗号プリミティブは実装・検証済みだが、完全な機密マルチノードラウンドトリップは明示的にPLANNED／ゲート付き）。
- アテステーション駆動型トラスト——ノードは何であることを証明しなければならない。SPIFFEアイデンティティ、GPUワークの証明、デバイスシーリング、輸出管理KYCゲート、EU AI法令準拠ドキュメント生成などにより、トラストはネットワーク上の位置ではなく、証拠によって得られる。
- コスト・炭素排出量を考慮したオーケストレーション——TCOモデリング、炭素排出量を考慮した配置、クラウドへのバースト、N+Kフェイルオーバーリザーブ、クラウドマーケットプレイスアダプタを備え、価格と排出量をスケジューリングの第一級入力として扱う。
- マルチ**Raft**コンセンサス——低レイテンシの一貫性を実現するため、シャードごとのRaftグループとリースホルダーによるローカル読み取りを採用。STONITHフェンシング（IPMI／EC2／Azure／SBD）により、ハングしたノードは確実に排除され、状態を破壊することはない。
- ドリフト防止のメカニカルリント——archlintは、ドキュメント化されたコンポーネントが存在しないパッケージパスにマッピングされた瞬間にビルドを失敗させる。また、ドキュメントチェーンエンジンにより、

Markdown／HTML／PDF／DOCXのバイト単位での一貫性を維持し、ドキュメントがコードに対して黙って嘘をつくことを防ぐ。

技術的課題とその解決策

- 「**PASS-bluff**」（機能しない機能でテストが通る問題）：プロジェクト全体が排除を目指す失敗モード。グリーンのテストスイートがスタブで通過する事態を防ぐため、全ての作業項目に「ガードテスト」を必須化。独立したコードミューテーションの下でこのテストが失敗することが確認されて初めて、作業完了とマークされる。これにより、テストがモックではなく実際の動作を検証していることが証明される。
- クロスプラットフォームの同等性（**Linux**専用モックの排除）：ビルドタグで分割された共通インターフェースを採用し、各OS固有の機能を活用。Linuxではcgroup・/proc・カーネルWireGuard、macOSではsysctl・vm_stat・IOKit・wireguard-goを使用。さらに独立したOSオラクルでクロスチェックを行い、各プラットフォームがLinuxの仮想的な状態ではなく、真のネイティブ状態を報告するようにした。
- 障害発生時の分散システムの正確性：決定論的シミュレーションテストとリニアラビリティチェッカーを導入。ネットワーク分断・クラッシュ・クロックスキューを人工的に発生させ、再現・検証を行う。TLA+の形式仕様にに基づき、コード実装前にコンセンサスとスケジューリングの不変条件を厳密に定義。
- ドキュメントとアーキテクチャの乖離：archlintを導入し、ドキュメントに記載されているが実際には存在しないパッケージがあればビルドを失敗させる。また、docs_chain検証ゲートを設け、例外なしで乖離をビルドエラーとして扱う。これにより、ドキュメントの陳腐化はウィキページの問題ではなく、システム全体の問題となる。
- 未完了作業の適切なスコープ設定：機密扱いのマルチノード推論ラウンドトリップは、チケットで明示的に管理し、「エンドツーエンドでの検証は未実施」とラベル付け。完成品のように見せかけるのではなく、未完了の作業にも完成した作業と同じ厳格な基準を適用。

テックスタック

- **Go (go.mod: 1.25 / ツールチェーン 1.26.4)**：約30モジュールのワークスペース全体で制御プレーン言語として採用。軽量なゴルーチンによる並行処理と、データセンターからエッジまで同一の静的バイナリでデプロイ可能な点が選定理由。
- **Zig (0.14+) + C/C++**：Goのランタイムが障害となる場面で採用。低レベルのシステムプリミティブや、ハードウェアを決定論的かつアロケーションフリーで制御する必要があるGPUカーネルに使用。
- **gRPC + Protocol Buffers**：全てのサブシステム間API (api/v1/)は型付きのバージョン管理された契約として定義。これにより、14のマイクロサービスが互いに破壊することなく進化し、独自のワイヤフォーマットを手作業で実装する必要がなくなる。
- **Raft (etcd-raft) + SWIM**ゴシップ：意図的な分離を採用。Raftは強い一貫性が求められる状態を管理し、SWIMゴシップはスケーラビ

リティが重要なメンバーシップやディスカバリーを担当。コンセンサスのオーバーヘッドを回避。

- **PostgreSQL 16、Redis 7クラスター、etcd v3.5、SQLite**: 用途に応じた適切なストレージを選択。Postgresは耐久性のあるリレーショナルデータ、Redisはホットキャッシュ、etcdは調整用、組み込みSQLiteはノードローカルなHXC作業項目レジストリとして使用。
- **NATS 2.10 (JetStream)、Kafka 4.0 (KRaft)、RabbitMQ 3.13**: 3種類のメッセージング基盤をトラフィックの特性に応じて使い分け。NATS/JetStreamは高速な内部イベント処理、Kafkaは耐久性のある高スループットストリーム、RabbitMQは従来型のブローカーセマンティクスに対応。
- **WireGuardメッシュ + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF**: WireGuardをノード間の軽量メッシュとして採用。ハイブリッドな耐量子鍵交換とAEADレコードでラップし、古典的および量子的な攻撃者に対する機密性を確保。
- **SPIFFE + JWT (HS256) + スコープベースRBAC + OPA**: 多層的なアイデンティティと認可を実現。SPIFFEはワークロードアイデンティティ、JWTはトークン、スコープベースRBACは大まかなアクセス制御、OPAはコードとしての細粒度ポリシーを担当。
- **Prometheus v2.50、Grafana 10.4、Jaeger 1.55、W3C** トレーシング: メトリクス、ダッシュボード、分散トレースをW3Cコンテキスト伝播で統合。リクエストをサービスやハードウェア層をまたいで追跡可能に。
- **HashiCorp Vault 1.16**: シークレットや鍵情報をコードや設定ファイルから分離し、監査可能な形で管理。
- **Docker Compose、Kubernetes (kustomize、セキュリティ強化されたsecurityContext)、Helm**: ローカル環境の立ち上げにはComposeを使用し、本番環境にはセキュリティ強化されたsecurityContextを持つKubernetes/Helmを採用。環境間で同一の定義を昇格させる。
- **React + TypeScript + Vite (Node 20+)**: セッション、ターミナル、プール利用状況を管理する高速かつ型安全なウェブUI。
- **TLA+**: コンセンサスとスケジューリングの不変条件を形式仕様として定義。実装前に最もテストが困難な特性を設計段階で証明。

内容

開発状況と注意事項

- **開発状況**: 進行中。現在のバージョンは初期段階 (0.1.0-dev) です。機密性を保持したマルチノード推論の完全なラウンドトリップ、マーケットプレイスの決済、アテストーションに基づくスケジューリングの実装など、複数の高度な機能はリポジトリ内で「PLANNED / infra-gated」と明示されており、完全な動作は保証されていません。カバレッジの数値は自己申告によるものです。
- **ライセンス**: 未定。明確な宣言はありません。HelmチャートのHelixCluster/HelixClusterおよびhelixcluster.ioのURLは未検証のプレースホルダーであり、実際のリモートとは一致しません。

- バンドルされているLLMスタックプロジェクト（LLMOrchestrator、LLMProvider、LLMsVerifier）は、クラスター内でホストされるモデルサーバーではなく、独立したサブモジュールです。

優先度レベル: Helix-primary（LLM-infrastructureクラスター — 推論およびコンピュートワークロードをホスト可能なコンピュート基盤）。HelixTrackに次ぐ優先順位です。